Gemini 4 Argon is now available. Try it out (https://aistudio.google.com/prompts/new_chat?model=gemini-4-argon).

# Embeddings

The Gemini API offers embedding models to generate embeddings for text, images, video, and other content. These resulting embeddings can then be used for tasks such as semantic search, classification, and clustering, providing more accurate, context-aware results than keyword-based approaches.

The latest model, **`gemini-embedding-2`**, is the first multimodal embedding model in the Gemini API. It maps text, images, video, audio, and documents into a unified embedding space, enabling cross-modal search, classification, and clustering across over 100 languages. See the multimodal embeddings section (#multimodal) to learn more. For text-only use cases, **`gemini-embedding-001`** remains available.

Building Retrieval Augmented Generation (RAG) systems is a common use case for AI products. Embeddings play a key role in significantly enhancing model outputs with improved factual accuracy, coherence, and contextual richness. If you prefer to use a managed RAG solution, we built the File Search (/gemini-api/docs/file-search) tool which makes doing RAG easier to manage and more cost effective.

## Generating embeddings

Use the **`embedContent`** method to generate text embeddings:

Python JavaScript (#javascript) Java (#java) Go (#go) REST (#rest)
(#python)

```
from google import genai

client = genai.Client()

result = client.models.embed_content(
        model="gemini-embedding-2",
        contents="What is the meaning of life?"
)

print(result.embeddings)
```

**Note:** While **`gemini-embedding-001`** lets you generate individual embeddings for a list of strings, Gemini Embedding 2 produces a single aggregated embedding for multiple inputs. You can find more information in Embedding aggregation (#embedding-aggregation), or you can use the Batch API (/gemini-api/docs/batch-api#batch-embedding) if you want to generate multiple embeddings at once.

## Specify task type to improve performance

You can use embeddings for a wide range of tasks from classification to document search. Specifying the right task type helps optimize the embeddings for the intended relationships, maximizing accuracy and efficiency.

### Task types with Embeddings 2

For text-only tasks with **`gemini-embedding-2`**, we strongly recommend you add the task instruction in your prompt. This can be done by formatting the query and the document with the correct task prefix.

When generating a single embedding based on multimodal input (#embedding-aggregation), we generally don't recommend prefixing the text portion of the input with a task instruction. In some cases it improves performance, but in others it reduces performance.

The following tables show examples of how to format queries and documents for symmetric and asymmetric use cases using the **`gemini-embedding-2`** model.

**Retrieval use cases (Asymmetric format)**

In asymmetric use cases, add the task prefix to the query and apply the document structure for the content you want to embed and retrieve.

| Use case | Query structure | Document structure |
|---|---|---|
| Search query | `task: search result \| query: {content}` | `title: {title} \| text: {content}`<br>If there is no title, use `title: none`. |
| Question answering | `task: question answering \| query: {content}` | `title: {title} \| text: {content}` |
| Fact checking | `task: fact checking \| query: {content}` | `title: {title} \| text: {content}` |
| Code retrieval | `task: code retrieval \| query: {content}` | `title: {title} \| text: {content}` |

**Example usage**

Python
(#python)

```
# Generate embedding for a task's query. Use your correct task here:
def prepare_query(query):
    # return f"task: question answering | query: {query}"
    # return f"task: fact checking | query: {query}"
    # return f"task: code retrieval | query: {query}"
    return f"task: search result | query: {query}"

# Generate embedding for document of an asymmetric retrieval task:
def prepare_document(content, title=None):
    if title is None:
        title = "none"
    return f"title: {title} | text: {content}"
```

**Single-input use cases (Symmetric format)**

In symmetric use cases, for the same task, use the same formatting for the query and the document.

| Use case | Input structure |
|---|---|
| Classification | `task: classification \| query: {content}` |
| Clustering | `task: clustering \| query: {content}` |
| Semantic similarity | `task: sentence similarity \| query: {content}`<br>Do not use this for search or retrieval. It is intended for semantic textual similarity. |

**Example usage**

Python
(#python)

```
# Generate embedding for query & document of your task.
def prepare_query_and_document(content):
    # return f'task: clustering | query: {content}'
    # return f'task: sentence similarity | query: {content}'
    return f'task: classification | query: {content}'
```

It is important that the task is used consistently. E.g., if documents are embedded with `f'task: classification | query: {content}'`, the query should also be embedded following this task format.

## Task types with Embeddings 1

For `gemini-embedding-001`, you can specify the `task_type` in the `embedContent` method. For a complete list of supported task types, see the Supported task types (#supported-task-types) table.

**Note:** You cannot use the `task_type` field for the `gemini-embedding-2` model. Instead, include the task as an instruction in your prompt, as detailed in the above section (#task-types-embeddings-2).

The following example shows how you can use `SEMANTIC_SIMILARITY` to check how similar in meaning strings of texts are.

Python JavaScript (#javascript) Java (#java) Go (#go) REST (#rest)
(#python)

```
from google import genai
from google.genai import types
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

client = genai.Client()

texts = [
    "What is the meaning of life?",
    "What is the purpose of existence?",
    "How do I bake a cake?",
]

result = client.models.embed_content(
    model="gemini-embedding-001",
    contents=texts,
    config=types.EmbedContentConfig(task_type="SEMANTIC_SIMILARITY")
)

# Create a 3x3 table to show the similarity matrix
df = pd.DataFrame(
    cosine_similarity([e.values for e in result.embeddings]),
    index=texts,
    columns=texts,
)

print(df)
```

The code snippets will show how similar the different chunks of text are to one another when run.

**Note:** Cosine similarity is a good distance metric because it focuses on direction rather than magnitude, which more accurately reflects conceptual closeness. Values range from -1 (opposite) to 1 (greatest similarity).

### Supported task types

Supported task types for `gemini-embedding-001`:

| Task type | Description | Examples |
|---|---|---|
| SEMANTIC_SIMILARITY | Embeddings optimized to assess text similarity. | Recommendation systems, duplicate detection |
| CLASSIFICATION | Embeddings optimized to classify texts according to preset labels. | Sentiment analysis, spam detection |
| CLUSTERING | Embeddings optimized to cluster texts based on their similarities. | Document organization, market research, anomaly detection |
| RETRIEVAL_DOCUMENT | Embeddings optimized for document search. | Indexing articles, books, or web pages for search. |

| | | |
|---|---|---|
| RETRIEVAL_QUERY | Embeddings optimized for general search queries. Use `RETRIEVAL_QUERY` for queries; `RETRIEVAL_DOCUMENT` for documents to be retrieved. | Custom search |
| CODE_RETRIEVAL_QUERY | Embeddings optimized for retrieval of code blocks based on natural language queries. Use `CODE_RETRIEVAL_QUERY` for queries; `RETRIEVAL_DOCUMENT` for code blocks to be retrieved. | Code suggestions and search |
| QUESTION_ANSWERING | Embeddings for questions in a question-answering system, optimized for finding documents that answer the question. Use `QUESTION_ANSWERING` for questions; `RETRIEVAL_DOCUMENT` for documents to be retrieved. | Chatbox |
| FACT_VERIFICATION | Embeddings for statements that need to be verified, optimized for retrieving documents that contain evidence supporting or refuting the statement. Use `FACT_VERIFICATION` for the target text; `RETRIEVAL_DOCUMENT` for documents to be retrieved | Automated fact-checking systems |

## Controlling embedding size

Both `gemini-embedding-001` and `gemini-embedding-2` are trained using the Matryoshka Representation Learning (MRL) technique which teaches a model to learn high-dimensional embeddings that have initial segments (or prefixes) which are also useful, simpler versions of the same data.

Use the `output_dimensionality` parameter to control the size of the output embedding vector. Selecting a smaller output dimensionality can save storage space and increase computational efficiency for downstream applications, while sacrificing little in terms of quality. By default, both models output a 3072-dimensional embedding, but you can truncate it to a smaller size without losing quality to save storage space. We recommend using 768, 1536, or 3072 output dimensions.

Python JavaScript (#javascript) Java (#java) Go (#go) REST (#rest)
(#python)

```
from google import genai
from google.genai import types

client = genai.Client()

result = client.models.embed_content(
    model="gemini-embedding-2",
    contents="What is the meaning of life?",
    config=types.EmbedContentConfig(output_dimensionality=768)
)

[embedding_obj] = result.embeddings
embedding_length = len(embedding_obj.values)

print(f"Length of embedding: {embedding_length}")
```

Example output from the code snippet:

```
Length of embedding: 768
```

## Ensuring quality for smaller dimensions

**Note:** `gemini-embedding-2` introduces automatic renormalization for non-default dimensions.

While the default 3072-dimension embeddings are always normalized, Gemini Embedding 2 also auto-normalizes truncated dimensions (e.g., 768, 1536). This ensures semantic similarity is calculated via vector direction rather than magnitude, providing more accurate results out of the box.

**Older Models**: If you are using `gemini-embedding-001`, you must manually normalize non-3072 dimensions as follows:

Python
(#python)

```
import numpy as np
from numpy.linalg import norm

# Only for embeddings from `gemini-embedding-001`
embedding_values_np = np.array(embedding_obj.values)
normed_embedding = embedding_values_np / np.linalg.norm(embedding_values_np)

print(f"Normed embedding length: {len(normed_embedding)}")
print(f"Norm of normed embedding: {np.linalg.norm(normed_embedding):.6f}") # Should be very close to 1
```

Example output from this code snippet:

```
Normed embedding length: 768
Norm of normed embedding: 1.000000
```

The following table shows the MTEB scores, a commonly used benchmark for embeddings, for different dimensions. Notably, the result shows that performance is not strictly tied to the size of the embedding dimension, with lower dimensions achieving scores comparable to their higher dimension counterparts.

| MRL Dimension | MTEB Score (Gemini Embedding 001) |
|---|---|
| 2048 | 68.16 |
| 1536 | 68.17 |
| 768 | 67.99 |
| 512 | 67.55 |
| 256 | 66.19 |
| 128 | 63.31 |

## Multimodal embeddings

The `gemini-embedding-2` model supports multimodal input, allowing you to embed images, video, audio, and documents content alongside text. All modalities are mapped into the same embedding space, enabling cross-modal search and comparison.

### Supported modalities and limits

The overall maximum input tokens limit is 8192 tokens.

| Modality | Specifications and limits |
|---|---|
| **Text** | Supports up to 8,192 tokens. |
| **Image** | Maximum of 6 images per request. Supported formats: PNG, JPEG. |
| **Audio** | Maximum duration of 180 seconds. Supported formats: MP3, WAV. |

| | |
|---|---|
| **Video** | Maximum duration of 120 seconds. Supported formats: MP4, MOV. Supported codecs: H264, H265, AV1, VP9.<br>The system processes a maximum of 32 frames per video: short videos (≤32s) are sampled at 1 fps, while longer videos are uniformly sampled to 32 frames. Audio tracks are not processed in video files. |
| **Documents (PDF)** | Maximum of 1 file per request, up to 6 pages. |

## Embedding images

The following example shows how to embed an image using `gemini-embedding-2`.

Images can be provided as inline data or as uploaded files through the Files API (/gemini-api/docs/files).

Python JavaScript (#javascript) Java (#java) REST (#rest)
(#python)

```
from google import genai
from google.genai import types

with open('example.png', 'rb') as f:
    image_bytes = f.read()

client = genai.Client()

result = client.models.embed_content(
    model='gemini-embedding-2',
    contents=[
        types.Part.from_bytes(
            data=image_bytes,
            mime_type='image/png',
        ),
    ]
)

print(result.embeddings)
```

## Embedding aggregation

When working with multimodal content, how you structure your input affects the embedding output:

- **Multiple parts (aggregated):** Adding multiple inputs directly to the `contents` parameter produces one aggregated embedding for all inputs.
- **Multiple `Content` objects (separate):** Wrapping each input in a `Content` object and passing them in the `contents` parameter returns separate embeddings for each entry.
- **Post-level representation:** For complex objects like social media posts with multiple media items, we recommend aggregating separate embeddings (for example, by averaging) to create a coherent post-level representation.

The following example shows how to create one aggregated embedding for text and image input. Simply add multiple inputs to the `contents` parameter:

Python JavaScript (#javascript) Java (#java) REST (#rest)
(#python)

```
from google import genai
from google.genai import types

client = genai.Client()

with open('dog.png', 'rb') as f:
    image_bytes = f.read()

result = client.models.embed_content(
    model='gemini-embedding-2',
```

```
    contents=[
        "An image of a dog",
        types.Part.from_bytes(
            data=image_bytes,
            mime_type='image/png',
        ),
    ]
)

# This produces one embedding
for embedding in result.embeddings:
    print(embedding.values)
```

**Note:** The text portion of the multimodal input shouldn't include task type information.

On the other hand, if you use `Content` objects inside the `contents` parameter, it returns separate embeddings. This example creates multiple embeddings in one embedding call:

Python JavaScript (#javascript) Java (#java) REST (#rest)
(#python)

```
from google import genai
from google.genai import types

client = genai.Client()

with open('dog.png', 'rb') as f:
    image_bytes = f.read()

result = client.models.embed_content(
    model="gemini-embedding-2",
    contents=[
        types.Content(parts=[types.Part.from_text(text="task: classification | query: An image of a dog")]),
        types.Content(
            parts=[
                types.Part.from_bytes(
                    data=image_bytes,
                    mime_type="image/png",
                ),
            ]
        ),
    ],
)

# This produces two embeddings
for embedding in result.embeddings:
    print(embedding.values)
```

## Embedding audio

The following example shows how to embed an audio file using `gemini-embedding-2`.

Audio files can be provided as inline data or as uploaded files through the Files API (/gemini-api/docs/files).

Python JavaScript (#javascript) Java (#java) REST (#rest)
(#python)

```
from google import genai
from google.genai import types

with open('example.mp3', 'rb') as f:
    audio_bytes = f.read()

client = genai.Client()

result = client.models.embed_content(
```

```
    model='gemini-embedding-2',
    contents=[
        types.Part.from_bytes(
            data=audio_bytes,
            mime_type='audio/mpeg',
        ),
    ]
)

print(result.embeddings)
```

## Embedding video

The following example shows how to embed a video using `gemini-embedding-2`.

Videos can be provided as inline data or as uploaded files through the Files API (/gemini-api/docs/files).

Python JavaScript (#javascript) Java (#java) REST (#rest)
(#python)

```
from google import genai
from google.genai import types

client = genai.Client()

with open('example.mp4', 'rb') as f:
    video_bytes = f.read()

result = client.models.embed_content(
    model='gemini-embedding-2',
    contents=[
        types.Part.from_bytes(
            data=video_bytes,
            mime_type='video/mp4',
        ),
    ]
)

print(result.embeddings[0].values)
```

If you need to embed videos >120 seconds, you can chunk the video into overlapping segments and embed those chunks individually.

## Embedding documents

Documents in PDF format can be embedded directly. The model processes the visual and text content of each page.

PDFs can be provided as inline data or as uploaded files through the Files API (/gemini-api/docs/files).

#### How the model processes PDFs

When you embed a PDF, the model processes the document using both visual and text features:

- **Visual representation:** The model renders each page as an image, which consumes **258 tokens** per page.
- **Text extraction:** The model extracts text from the document. For **native PDFs** (which contain digital text), the model extracts the text directly. For **scanned PDFs** (which contain images of text), the model automatically runs optical character recognition (OCR) to extract the text.

To calculate the total token count for a PDF, add the visual tokens (258 per page) to the text tokens. Your inputs must fit within the model's **8,192 token limit** (shared across all modalities). The system silently truncates inputs that exceed this limit.

**Note:** The Gemini Developer API always enables OCR for PDFs and does not support disabling it (the `document_ocr` parameter is only available in Vertex AI).

#### PDF limits

- **Files per request:** You can submit a maximum of 1 PDF file.
- **Page limit:** You can submit a maximum of 6 pages per file. For the best quality, we strongly recommend using 1 page per PDF.

The following example shows how to embed a PDF using `gemini-embedding-2`:

Python JavaScript (#javascript) Java (#java) REST (#rest)
(#python)

```
from google import genai
from google.genai import types

with open('example.pdf', 'rb') as f:
    pdf_bytes = f.read()

client = genai.Client()

result = client.models.embed_content(
    model='gemini-embedding-2',
    contents=[
        types.Part.from_bytes(
            data=pdf_bytes,
            mime_type='application/pdf',
        ),
    ]
)

print(result.embeddings)
```

## Use cases

Text embeddings are crucial for a variety of common AI use cases, such as:

- **Retrieval-Augmented Generation (RAG):** Embeddings enhance the quality of generated text by retrieving and incorporating relevant information into the context of a model.
- **Information retrieval:** Search for the most semantically similar text or documents given a piece of input text.

  Document search tutorial (https://github.com/google-gemini/cookbook/blob/main/examples/Talk_to_documents_with_embeddings.ipynb)
- **Search reranking**: Prioritize the most relevant items by semantically scoring initial results against the query.

  Search reranking tutorial (https://github.com/google-gemini/cookbook/blob/main/examples/Search_reranking_using_embeddings.ipynb)
- **Anomaly detection:** Comparing groups of embeddings can help identify hidden trends or outliers.

  Anomaly detection tutorial (https://github.com/google-gemini/cookbook/blob/main/examples/Anomaly_detection_with_embeddings.ipynb)
- **Classification:** Automatically categorize text based on its content, such as sentiment analysis or spam detection

  Classification tutorial (https://github.com/google-gemini/cookbook/blob/main/examples/Classify_text_with_embeddings.ipynb)
- **Clustering:** Effectively grasp complex relationships by creating clusters and visualizations of your embeddings.

  Clustering visualization tutorial (https://github.com/google-gemini/cookbook/blob/main/examples/clustering_with_embeddings.ipynb)

## Storing embeddings

As you take embeddings to production, it is common to use **vector databases** to efficiently store, index, and retrieve high-dimensional embeddings. Google Cloud offers managed data services that can be used for this purpose including Gemini Enterprise Agent Platform Vector Search 2.0 (https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/vector-search-2/overview), BigQuery (https://cloud.google.com/bigquery/docs/introduction), AlloyDB (https://cloud.google.com/alloydb/docs/overview), and Cloud SQL (https://cloud.google.com/sql/docs/postgres/introduction).

The following tutorials show how to use other third party vector databases with Gemini Embedding.

- **ChromaDB tutorials** (https://docs.trychroma.com/integrations/embedding-models/google-gemini)
- **QDrant tutorials** (https://qdrant.tech/documentation/embeddings/gemini/)
- **Weaviate tutorials** (https://docs.weaviate.io/weaviate/model-providers/google)
- **Pinecone tutorials** (https://github.com/google-gemini/cookbook/blob/main/examples/langchain/Gemini_LangChain_QA_Pinecone_WebLoad.ipynl

## Model versions

Gemini Embedding 2Gemini Embedding...
(#gemini-embedding-2)

| Property | Description | |
|---|---|---|
| Model code | **Gemini API**<br>`gemini-embedding-2` | |
| Supported data types | **Input**<br>Text, image, video, audio, PDF | **Output**<br>Text embeddings |
| Token limits[*] (/gemini-api/docs/tokens) | **Input token limit**<br>8,192 | **Output dimension size**<br>Flexible, supports: 128 - 3072,<br>Recommended: 768, 1536, 3072 |
| Versions | Read the model version patterns (/gemini-api/docs/models/gemini#model-versions) for more details.<br>• Stable: `gemini-embedding-2` | |
| Latest update | April 2026 | |

For deprecated Embeddings models, visit the Deprecations (/gemini-api/docs/deprecations) page

## Migration from gemini-embedding-001

The embedding spaces between `gemini-embedding-001` and `gemini-embedding-2` are **incompatible**. This means you cannot directly compare embeddings generated by one model with embeddings generated by the other. If you are upgrading to `gemini-embedding-2`, you must re-embed all of your existing data.

Beyond incompatibility, there are several other notable differences between the two models:

- **Task type specification:** With `gemini-embedding-001`, you specify the task type using the `task_type` parameter (e.g., `SEMANTIC_SIMILARITY`, `RETRIEVAL_DOCUMENT`). With `gemini-embedding-2`, the `task_type` parameter is not supported. Instead, you should include task instructions directly in the prompt for text-only tasks. See Task types with Embeddings 2 (#task-types-embeddings-2) for details on how to format prompts for different use cases.
- **Embedding aggregation:** `gemini-embedding-001` generates individual embeddings for each string in a list of inputs. In contrast, `gemini-embedding-2` produces a single, aggregated embedding when multiple inputs (like text and images) are provided directly in one request. To generate separate embeddings for individual inputs, wrap each input in a `Content` object, or use the Batch API (/gemini-api/docs/batch-api#batch-embedding). See Embedding aggregation (#embedding-aggregation) for more information.
- **Normalization:** If you use `output_dimensionality` to request embeddings with fewer than 3072 dimensions, `gemini-embedding-2` automatically normalizes these truncated embeddings. With `gemini-embedding-001`, you need to perform manual normalization for dimensions other than 3072. See Ensuring quality for smaller dimensions (#quality-for-smaller-dimensions) for details.

## Batch embeddings

If latency is not a concern, try using the Gemini Embeddings models with Batch API (/gemini-api/docs/batch-api#batch-embedding). This allows for much higher throughput at 50% of the default Embedding price. Find examples on how to get started in the Batch API cookbook (https://github.com/google-gemini/cookbook/blob/main/quickstarts/Batch_mode.ipynb).

## Responsible use notice

Unlike generative AI models that create new content, the Gemini Embedding model is only intended to transform the format of your input data into a numerical representation. While Google is responsible for providing an embedding model that transforms the format of your input data to the numerical-format requested, users retain full responsibility for the data they input and the resulting embeddings. By using the Gemini Embedding model you confirm that you have the necessary rights to any content that you upload. Do not generate content that infringes on others' intellectual property or privacy rights. Your use of this service is subject to our Prohibited Use Policy (https://policies.google.com/terms/generative-ai/use-policy) and Google's Terms of Service (/gemini-api/terms).

## Start building with embeddings

Check out the embeddings quickstart notebook (https://github.com/google-gemini/cookbook/blob/main/quickstarts/Embeddings.ipynb) to explore the model capabilities and learn how to customize and visualize your embeddings.

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-10-09 UTC.